

# 第一次课后作业

## P17 选择题

### 1.2

- (1)D
- (2)C
- (3)C
- (4)A
- (5)D
- (6)D
- (7)C
- (8)D
- (9)D
- (10)C

## 简答题

### 1.4

**题目：**1.4 计算机系统从功能上可划分为哪些层次？各层次在计算机系统中起什么作用？

**解题过程与分析：**

计算机系统通常被视为一个层次结构的集合，每一层都建立在下一层的基础之上，通过抽象屏蔽了底层的复杂性。根据经典的计算机组成原理，我们可以将其划分为以下 **6 个级别**：

#### 1. 逻辑门层：

- **作用：**这是计算机系统的最底层硬件，由逻辑门（与、或、非门等）和触发器组成，构成了处理器、存储器等核心部件的物理基础。

#### 2. 微代码层：

- **作用：** 负责为指令集架构层提供机器指令的解释执行功能。它通过微指令序列来控制硬件路径，从而实现一条复杂的机器指令。

### 3. 指令集架构层：

- **作用：** 是软硬件系统的界面。它定义了机器语言程序员可见的指令集、寄存器和内存结构。用户可以通过编写机器语言程序直接实现对计算机硬件的控制。

### 4. 操作系统层：

- **作用：** 负责管理和调度计算机中的软件和硬件资源（如 CPU 调度、内存管理等）。它为上层软件提供了一个虚拟机环境，极大地方便了用户使用计算机。

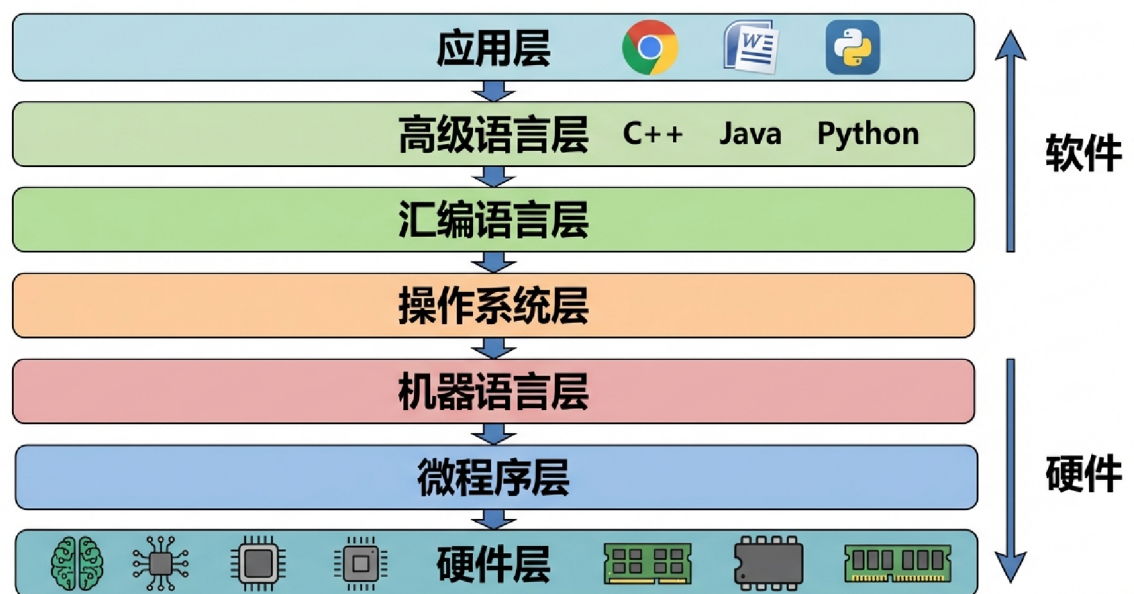
### 5. 汇编语言层：

- **作用：** 使用与机器硬件直接相关的汇编语言编写程序。汇编代码需要经过汇编程序转换成目标代码（机器代码）后，才能在机器上直接运行。

### 6. 高级语言层：

- **作用：** 这是面向用户的抽象层。程序员使用与机器无关的高级语言（如 C, Java, Python 等）编程。代码在编译器的作用下生成汇编代码或机器代码，极大地提高了开发效率和程序的可移植性。

## 计算机系统多级层次结构图



## 1.5

题目：

1.5 假定某计算机 1 和计算机 2 以不同的方式实现了相同的指令集，该指令集中共有 A、B、C、D 4 类指令，它们所占的比例分别为 40%、20%、15% 和 25%。计算机 1 和计算机 2 的时钟周期分别为 600MHz 和 800MHz，各类指令在两计算机上的 CPI 如表 1.8 所示。求两计算机的 MIPS 各为多少？

解题过程与分析：

### 1. 核心公式

- 平均 CPI 计算： $CPI = \sum_{i=1}^n (CPI_i \times \text{比例}_i)$
- MIPS 计算： $MIPS = \frac{f}{CPI \times 10^6}$  (其中  $f$  为时钟频率，单位为 Hz)

### 2. 计算过程

对于计算机 1:

- 计算  $CPI_1$ :

$$CPI_1 = 2 \times 0.4 + 3 \times 0.2 + 4 \times 0.15 + 5 \times 0.25 = 0.8 + 0.6 + 0.6 + 1.25 = 3.25$$

- 计算  $MIPS_1$ :

$$\text{已知 } f_1 = 600\text{MHz} = 600 \times 10^6 \text{Hz}$$

$$MIPS_1 = \frac{600 \times 10^6}{3.25 \times 10^6} \approx 184.62 \approx 185$$

对于计算机 2:

- 计算  $CPI_2$ :

$$CPI_2 = 2 \times 0.4 + 2 \times 0.2 + 3 \times 0.15 + 4 \times 0.25 = 0.8 + 0.4 + 0.45 + 1.0 = 2.65$$

- 计算  $MIPS_2$ :

$$\text{已知 } f_2 = 800\text{MHz} = 800 \times 10^6 \text{Hz}$$

$$MIPS_2 = \frac{800 \times 10^6}{2.65 \times 10^6} \approx 301.89 \approx 302$$

答：计算机 1 的 MIPS 约为 185，计算机 2 的 MIPS 约为 302。

## 1.6

题目：

1.6 若某程序编译后生成的目标代码由 A、B、C、D 4 类指令组成，它们在程序中所占比例分别为 40%、20%、15%、25%。已知 A、B、C、D 四类指令的 CPI 分别为 1、2、2、2。现需要对程序进行编译优化，优化后的程序中 A 类指令数量减少了一半，而其他指令数量未发生变化。假设运行该程序的计算机 CPU 主频为 500MHz。回答下列问题：

- (1) 优化前后程序的 CPI 各为多少？
- (2) 优化前后程序的 MIPS 各为多少？

(3) 通过上面的计算结果，你能得出什么结论？

### 解题过程与分析：

#### 1. 优化前后 CPI 的计算

- 优化前：

直接利用占比加权求和：

$$CPI_{\text{前}} = \sum(CPI_i \times \text{占比}_i) = 1 \times 0.4 + 2 \times 0.2 + 2 \times 0.15 + 2 \times 0.25 = 0.4 + 0.4 + 0.3 + 0.5 = 1.6$$

- 优化后：

假设原指令总数为  $N$ 。则 A 为  $0.4N$ ，B 为  $0.2N$ ，C 为  $0.15N$ ，D 为  $0.25N$ 。

优化后 A 减半变为  $0.2N$ ，其他不变。

$$\text{新指令总数 } IC_{\text{后}} = 0.2N + 0.2N + 0.15N + 0.25N = 0.8N。$$

**新占比：** A 为  $0.2/0.8 = 1/4$ ，B 为  $0.2/0.8 = 1/4$ ，C 为  $0.15/0.8 = 3/16$ ，D 为  $0.25/0.8 = 5/16$ 。

$$CPI_{\text{后}} = 1 \times \frac{1}{4} + 2 \times \frac{1}{4} + 2 \times \frac{3}{16} + 2 \times \frac{5}{16} = 0.25 + 0.5 + 0.375 + 0.625 = 1.75$$

#### 2. 优化前后 MIPS 的计算

已知主频  $f = 500\text{MHz}$ 。

- 优化前：

$$MIPS_{\text{前}} = \frac{f}{CPI_{\text{前}} \times 10^6} = \frac{500 \times 10^6}{1.6 \times 10^6} = 312.5$$

- 优化后：

$$MIPS_{\text{后}} = \frac{f}{CPI_{\text{后}} \times 10^6} = \frac{500 \times 10^6}{1.75 \times 10^6} \approx 285.7$$

#### 3. 结论分析

虽然优化后程序的 **CPI 变大了**，**MIPS 变小了**，但实际上由于指令总数减少了，程序的**总执行时间缩短了**。

- $T_{\text{前}} = \frac{N \times 1.6}{f}$

- $T_{\text{后}} = \frac{0.8N \times 1.75}{f} = \frac{1.4N}{f}$

显然  $T_{\text{后}} < T_{\text{前}}$ 。

通过本题计算可以发现，**MIPS 指标并不能简单地用来评判计算机性能**。在本例中，编译优化减少了执行代价最小的指令，导致平均每条指令的时钟周期上升，进而导致 MIPS 下降，但最终的程序运行速度反而提升了。这证明了评估性能时，程序的总执行时间才是最可靠的标准。

答：

(1) 优化前 CPI 为 1.6，优化后 CPI 为 1.75。

(2) 优化前 MIPS 为 312.5，优化后 MIPS 约为 285.7。

(3) 结论：不能简单地通过 CPI 或 MIPS 来评判性能，执行时间才是衡量性能的最终标准。

## P53 选择题

(1)B

(2)A

(3)B

(4)D

(5)A

(6)A

(7)D

(8)A

(9)A

(10)B

(11)C

## 简答题

### 2.3.(2)

**题目：**（2）相对于奇偶校验，交叉奇偶校验的检错与纠错能力的提高需要付出哪些方面的代价？

**解题过程与分析：**

交叉奇偶校验通过在水平和垂直两个方向上分别设置奇偶校验位，显著提升了检测多位错误和纠正单比特错误的能力。然而，这种能力的提升是以牺牲系统资源为代价的，主要包括以下几个方面：

#### 1. 编码效率降低：

- **分析：** 普通奇偶校验只需一个校验位。而交叉奇偶校验需要为每一行和每一列都增加一个校验位。
- **结果：** 冗余校验位的比例大幅增加，导致**编码效率**显著下降。

## 2. 硬件成本开销：

- **分析：** 奇偶校验的本质是异或运算。由于校验位数量增多，系统需要集成更多的**异或门电路**来实现并行计算。
- **结果：** 增加了芯片的面积需求和硬件逻辑的复杂程度。

## 3. 存储与带宽压力：

- **分析：** 更多的冗余位意味着在存储数据时需要更大的存储空间，在数据传输时会占用更多的总线带宽。

答：需要增加奇偶校验组，也就是需要增加更多的冗余校验位，这样会导致编码效率降低；另外编解码电路需要更多的异或门开销。

## 2.3.(8)

### 简答题

题目：

(8) 如何识别浮点数的正负？浮点数能表示的数值范围和数值的精度取决于什么？

解题过程与分析：

浮点数在计算机中的表示类似于科学计数法，通常由 **阶码** 和 **尾数** 两部分组成。

### 1. 正负判别：

- **机制：** 浮点数的正负由**尾数的符号位**决定。
- **标准：** 在广泛使用的 **IEEE 754 标准**中，浮点数的第一位即为符号位 ( $S$ )。
  - 若  $S = 0$ ，表示该浮点数为正数；
  - 若  $S = 1$ ，表示该浮点数为负数。

### 2. 数值范围：

- **决定因素：** 取决于**阶码的位数**。
- **分析：** 阶码反映了小数点在数中的实际位置。阶码的位数越多，能够表示的指数范围就越大，从而使浮点数能覆盖更大的绝对值区间（包括极大的数和极小的接近 0 的数）。

### 3. 数值精度：

- **决定因素：** 取决于**尾数的位数**。
- **分析：** 尾数代表了数的有效数字。尾数的位数越多，能够保留的有效位数就越丰富，计算结果的离散程度越小，即精度越高。

佐证资料：

根据浮点数通用公式： $N = (-1)^S \times M \times R^E$

- $S$ : 符号位，决定正负。
- $E$ : 阶码（以  $R$  为底），阶码位数决定  $N$  的范围。
- $M$ : 尾数，其位数决定有效数字的长度，即精度。

答：浮点数的正负由尾数的符号位决定，当采用 IEEE 754 格式时，通过数符就能判断出浮点数的正负。浮点数所能表示的数值范围和精确度分别取决于阶码的位数和尾数的位数。

## 2.3.(10)

题目：

(10) 简述 CRC 校验码的检错原理，CRC 能纠错吗？

解题过程与分析：

循环冗余校验（CRC）是一种根据网络数据包或计算机文件等信源产生简短固定位数校验码的检错方法。

### 1. 检错原理：

- 核心算法：采用模2除法。
- 过程：发送方和接收方约定一个共同的生成多项式  $G(x)$ 。发送部件在数据后面附加校验位形成 CRC 码。接收部件收到后，同样用  $G(x)$  对收到的码字进行模2除法运算。
- 判定：
  - 若余数为 0，表示数据在传输过程中高概率正确。
  - 若余数不为 0，则表示数据传输过程中发生了错误（出错）。

### 2. 纠错能力：

- 理论上：CRC 具备纠错能力。因为对于特定的生成多项式，不同位出错所产生的余数（也称为指纹或综合向量）通常是唯一的。通过查找“余数与出错位”的对应关系表，可以直接确定是哪一位出错了。
- 实际上：在计算机网络和存储系统中，CRC 主要用于检错。
- 原因：如果硬件要实现自动纠错，逻辑电路会变得非常复杂。相比之下，发现错误后要求发送方重发（ARQ）的成本更低且更可靠。

答：发送部件将某信息的 CRC 码传送至接收部件，接收部件收到 CRC 码后，仍用约定的生成多项式  $G(x)$  去除，若余数为 0，表示数据高概率正确；若余数不为 0，表示出错，再由余数的值来确定哪一位出错，从而加以纠正。由于 CRC 码不同位出错的余数均不相同，因此用户可以根据余数的值直接确定出错位，但由于 CRC 编码检错能力优异，因此它主要用于检错。

## 2.6

**题目:** C 语言中允许无符号数和有符号整数之间的转换, 下面是一段 C 语言代码。给出在 32 位计算机中上述程序段的输出结果并分析原因。

### 【输出结果】

```
x=4294967295=-1
u=2147483648=-2147483648
```

### 【过程与分析】

#### 1. 数据的机器码表示:

在 32 位计算机中, 整数以**补码**形式存储。

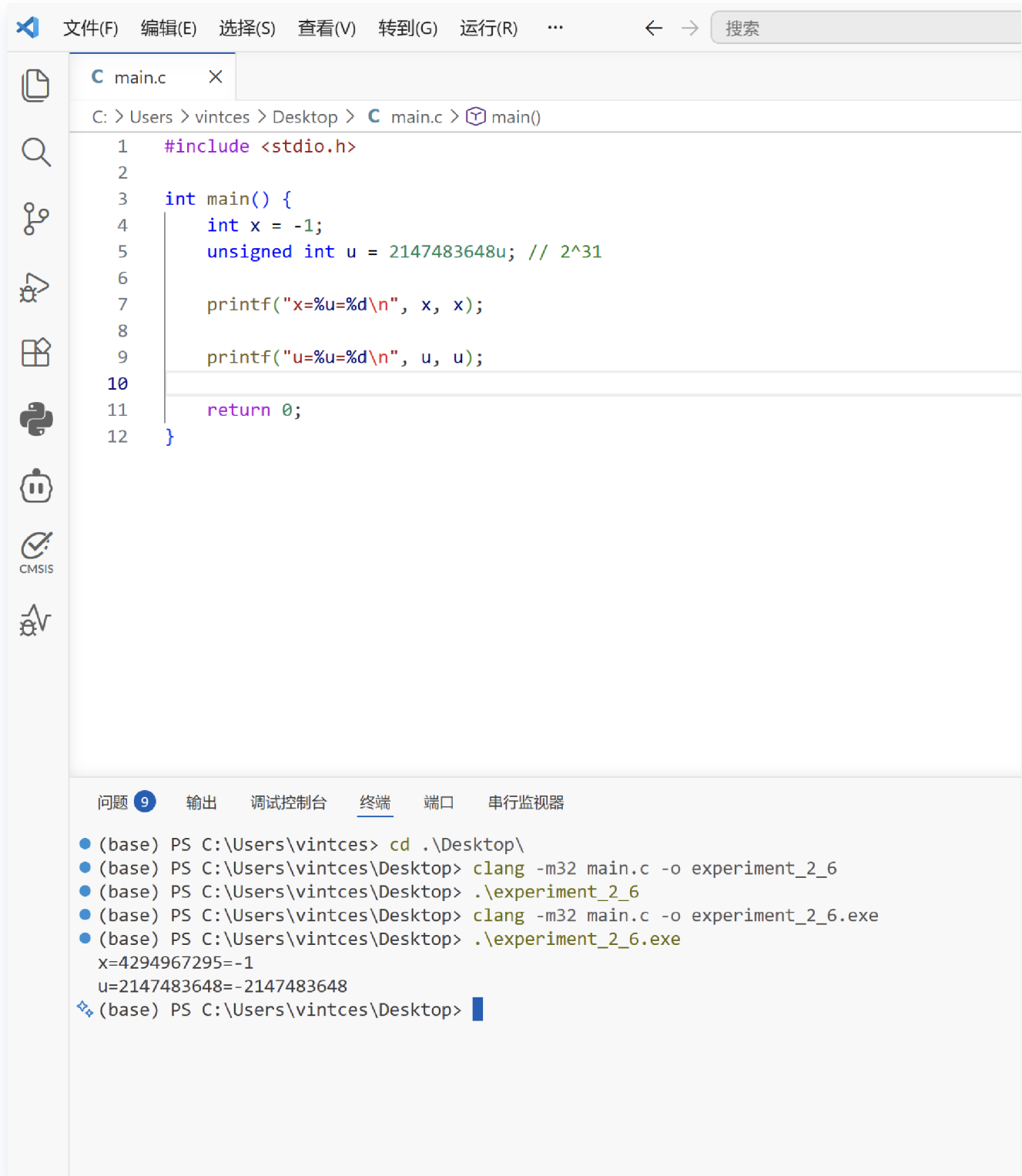
- 对于  $x = -1$ : 其原码为 `1000...0001`, 补码为全 `1`, 即 `0xFFFFFFFF`。
- 对于  $u = 2147483648$  ( $2^{31}$ ): 其二进制表示为首位为 `1`, 其余全为 `0`, 即 `1000...0000` (十六进制 `0x80000000`)。

#### 2. printf 格式化解析:

- 针对  $x$ :
  - 使用 `%u` 输出时, 系统将机器码 `0xFFFFFFFF` 视为无符号数, 计算结果为  $2^{32} - 1 = 4294967295$ 。
  - 使用 `%d` 输出时, 系统将其视为有符号补码, 解析回真值 `-1`。
- 针对  $u$ :
  - 使用 `%u` 输出时, 机器码 `0x80000000` 被视为无符号数, 即  $2^{31} = 2147483648$ 。
  - 使用 `%d` 输出时, 机器码 `0x80000000` 的首位被视为符号位, 根据补码规则, 该位模式对应的真值为  $-2^{31} = -2147483648$ 。

### 【资料佐证】





## 2.10

### 题目：

2.10 求与单精度浮点数 43940000H 对应的十进制数。

### 解题过程与分析：

本题考查的是 **IEEE 754 标准** 下单精度浮点数（32位）的解码过程。

### 1. 将十六进制数转换为二进制数

首先将  $43940000H$  展开为 32 位二进制格式：

$4 \rightarrow 0100$

$3 \rightarrow 0011$

$9 \rightarrow 1001$

$4 \rightarrow 0100$

后续 0 补齐，得到：

$(0\ 10000111\ 001010000000000000000000)_2$

### 2. 字段拆分与识别

根据 IEEE 754 标准，32 位单精度浮点数的结构如下：

- **符号位 (S)**：第 31 位。
- **阶码 (E)**：第 30-23 位（共 8 位），采用移码表示，偏置常数为 127。
- **尾数 (M)**：第 22-0 位（共 23 位），隐含最高位 1。
- **S = 0**：代表该数为**正数**。
- **阶码 E**：二进制为  $(10000111)_2$ 。

计算真值： $e = E - 127 = 135 - 127 = 8$ 。

- **尾数 M**：二进制部分为  $00101\dots$ ，加上隐含的最高位 1，得到：  
 $(1.00101)_2$ 。

### 3. 计算十进制值

利用公式： $V = (-1)^S \times (1.M) \times 2^e$

$$V = 1.00101 \times 2^8$$

将小数点向右移动 8 位：

$$(100101000)_2$$

转换为十进制：

$$2^8 + 2^5 + 2^3 = 256 + 32 + 8 = 296$$

**答：单精度浮点数  $43940000H$  对应的十进制数为 296。**

## 2.14

### 题目：

2.14 将下列十进制数表示成浮点规格化数，阶码为 4 位，尾数为 10 位，各含 1 位符号，阶码和尾数均用补码表示。

(1)  $57/128$ ； (2)  $-69/128$ 。

### 解题过程与分析：

本题要求按照特定的格式（阶码 4 位补码，尾数 10 位补码，均为规格化数）进行转换。

#### 1. 基础转换（十进制转二进制）

- (1)  $57/128$ ：

$$57 = 32 + 16 + 8 + 1 = (111001)_2$$

$$128 = 2^7$$

$$\text{所以 } 57/128 = (0.0111001)_2$$

- (2)  $-69/128$ ：

$$69 = 64 + 4 + 1 = (1000101)_2$$

$$\text{所以 } -69/128 = -(0.1000101)_2$$

#### 2. 规格化与阶码计算

对于补码规格化浮点数，尾数  $M$  的形式应满足：正数为  $0.1xxx\dots$ ，负数为  $1.0xxx\dots$ 。

- (1)  $57/128$  处理：

- 原码表示： $0.0111001 = 0.111001 \times 2^{-1}$
- 阶码：真值为  $-1$ 。4 位补码表示为 1111（计算： $10000 - 0001 = 1111$ ）。
- 尾数：真值为  $0.111001$ 。10 位补码表示为  $0.111001000$ （符号位占 1 位）。
- 最终结果：1111, 0111001000

- (2)  $-69/128$  处理：

- 二进制： $-(0.1000101)_2$ 。
- 尾数补码： $-(0.1000101)$  的补码为  $1.0111011$ 。
- 检查规格化：符号位与最高数值位不同（1.0），已是规格化数。
- 阶码：无需移动小数点，真值为 0。4 位补码为 0000。
- 尾数：10 位补码表示为  $1011101100$ （末尾补 0 凑齐位数）。
- 最终结果：0000, 1011101100

答：

- (1) 57/128 表示为：阶码 1111，尾数 0111001000。
- (2) -69/128 表示为：阶码 0000，尾数 1011101100。

2.17

题目：设 8 位有效信息为 01101110，试写出它的海明校验码。给出过程，说明分组检测方式，并给出指错字及其逻辑表达式。如果接收方收到的有效信息变成 01101111，说明如何定位错误并纠正错误。

1. 确定校验位位数

设有效信息位数为  $n = 8$ ，校验位位数为  $r$ 。

根据海明不等式： $2^r \geq n + r + 1$

将  $n = 8$  代入，当  $r = 4$  时， $2^4 = 16 \geq 8 + 4 + 1 = 13$ ，满足条件。

因此，海明码总位数为 12 位（用  $H_{12}$  到  $H_1$  表示）。

2. 说明分组检测方式与映射关系

将 8 位数据（ $D_8$  到  $D_1 = 01101110$ ）填入非  $2^k$  的位置，将校验位（ $P_4$  到  $P_1$ ）填入  $2^k$  的位置。分组方式由位置下标的二进制决定，位权相同的为一组：

| 海明码位置 | H12   | H11   | H10   | H9    | H8    | H7    | H6    | H5    | H4    | H3    | H2    | H1    |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 映射关系  | $D_8$ | $D_7$ | $D_6$ | $D_5$ | $P_4$ | $D_4$ | $D_3$ | $D_2$ | $P_3$ | $D_1$ | $P_2$ | $P_1$ |
| 有效信息值 | 0     | 1     | 1     | 0     | 待求    | 1     | 1     | 1     | 待求    | 0     | 待求    | 待求    |

3. 计算海明校验码

根据偶校验规则，各校验位的逻辑表达式及计算过程如下：

- $P_1 = D_7 \oplus D_5 \oplus D_4 \oplus D_2 \oplus D_1 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1$
- $P_2 = D_7 \oplus D_6 \oplus D_4 \oplus D_3 \oplus D_1 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 = 0$
- $P_3 = D_8 \oplus D_4 \oplus D_3 \oplus D_2 = 0 \oplus 1 \oplus 1 \oplus 1 = 1$
- $P_4 = D_8 \oplus D_7 \oplus D_6 \oplus D_5 = 0 \oplus 1 \oplus 1 \oplus 0 = 0$

将求得的校验位代入映射表，得到完整的海明校验码为：

$D_8D_7D_6D_5P_4D_4D_3D_2P_3D_1P_2P_1 = 011001111001$

4. 接收端指错字计算与查表纠错过程

当接收方收到的有效信息由 01101110 变成 01101111 时，意味着接收到的完整海明码变成了 011001111101（即  $D_1$  位由 0 变为了 1）。

(1) 计算指错字及其逻辑表达式

指错字设为  $G_4G_3G_2G_1$ ，通过接收到的海明码重新进行偶校验计算：

- $G_1 = P_1 \oplus D_7 \oplus D_5 \oplus D_4 \oplus D_2 \oplus D_1 = 1 \oplus 1 \oplus 0 \oplus 1 \oplus 1 \oplus 1 = 1$
- $G_2 = P_2 \oplus D_7 \oplus D_6 \oplus D_4 \oplus D_3 \oplus D_1 = 0 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 1 = 1$
- $G_3 = P_3 \oplus D_8 \oplus D_4 \oplus D_3 \oplus D_2 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 1 = 0$
- $G_4 = P_4 \oplus D_8 \oplus D_7 \oplus D_6 \oplus D_5 = 0 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 0$

计算得出指错字 $G_4G_3G_2G_1 = 0011$ 。

(2) 根据余数确定出错位的完整方法（纠错对照表）

根据提供的查表法逻辑，以下是 12 位海明码全部可能状态的完整映射表：

| 指错字 (余数) G4G3G2G1 | 确定出错位置   | 具体出错位解析                      |
|-------------------|----------|------------------------------|
| 0000              | 没有错误     | 数据传输完全正确                     |
| 0001              | 第 1 位错误  | 海明码 $H_1$ 出错（校验位 $P_1$ 错）    |
| 0010              | 第 2 位错误  | 海明码 $H_2$ 出错（校验位 $P_2$ 错）    |
| 0011              | 第 3 位错误  | 海明码 $H_3$ 出错（数据位 $D_1$ 错）    |
| 0100              | 第 4 位错误  | 海明码 $H_4$ 出错（校验位 $P_3$ 错）    |
| 0101              | 第 5 位错误  | 海明码 $H_5$ 出错（数据位 $D_2$ 错）    |
| 0110              | 第 6 位错误  | 海明码 $H_6$ 出错（数据位 $D_3$ 错）    |
| 0111              | 第 7 位错误  | 海明码 $H_7$ 出错（数据位 $D_4$ 错）    |
| 1000              | 第 8 位错误  | 海明码 $H_8$ 出错（校验位 $P_4$ 错）    |
| 1001              | 第 9 位错误  | 海明码 $H_9$ 出错（数据位 $D_5$ 错）    |
| 1010              | 第 10 位错误 | 海明码 $H_{10}$ 出错（数据位 $D_6$ 错） |
| 1011              | 第 11 位错误 | 海明码 $H_{11}$ 出错（数据位 $D_7$ 错） |
| 1100              | 第 12 位错误 | 海明码 $H_{12}$ 出错（数据位 $D_8$ 错） |
| 1101 到 1111       | 无法映射     | 出现多位错误或其他异常状态                |

(3) 定位与纠正结论

查上面的对照表可知，余数为 0011 严格对应第 3 位错误，即海明码的  $H_3$  位发生翻转。

因为  $H_3$  映射的是有效信息位  $D_1$ ，由此定位到是  $D_1$  位出错。

**纠正错误的方法：**将接收器接收到的  $D_1$  位的值 (1) 进行硬件逻辑上的**取反操作**，使其恢复为正确的 0，即可完成纠错。

## 2.18

**题目：**

设要采用 **CRC 码**（循环冗余校验码）传送数据信息  $x = 1001$ ，当生成多项式为  $G(x) = 1101$  时，请写出它的循环冗余校验码。若接收方收到的数据信息为  $x' = 1101$ ，说明如何定位错误并纠正错误。

**解题过程：**

### 1. 求 CRC 校验码

- **确定校验位位数：**生成多项式  $G(x) = 1101$  为 4 位，其阶数  $k = 3$ （最高幂次为 3）。因此需要在信息位后面补 3 个 0。
- **模 2 除法：**将补 0 后的信息位 1001000 与生成多项式 1101 进行模 2 除法运算。
  - 计算式： $\frac{M(x) \cdot x^3}{G(x)} = \frac{1001000}{1101}$
  - 得到商为 1111，余数为 011。
- **得出结果：**将余数替换掉补位，得到最终的循环冗余校验码为 1001011。

### 2. 错误定位与纠正

- **接收端校验：**接收方收到的信息  $x' = 1101$ ，结合校验位补全后，接收到的完整码字为 1101011。
- **进行模 2 除法：**
  - $\frac{1101011}{1101} = 1000 \dots 011$ （商为 1000，余数为 011）。
- **循环纠错分析：**

由于余数不为 0，说明传输中发生了错误。根据 CRC 循环纠错特性，将余数 011 继续补 0 做模 2 除法：

  - 第一次补 0 后除法余数为 110。
  - 第二次补 0 后除法余数为 001。
- **结论：**经过两次运算后余数为 001，这表明出错位置在左侧起第 2 位（即权重最高的第 2 位）。
- **纠正方式：**将左侧起第 2 位取反（由 1 变为 0），即可将数据恢复为正确的 1001011。此外，在实际工程中也可以通过预先计算的“余数-错误位置”对应表（查表法）来快速定位。